

Построение пайплайнов данных

Мониторинг

Малахов Дмитрий Андреевич
Новосельцев Максим Сергеевич

23 января 2026 года

Зачем нужен мониторинг

- Оповещения в реальном времени
- Оптимизация управление рабочими нагрузками
- Упрощение устранения неполадок
- Четко представлять расходы
- Доступ к аналитике



Метрики vs логи

Логи — это про точность: мы собираем информацию на каждое событие. Если их агрегировать, можно увидеть общую картину и даже вытащить детали вплоть до единичного запроса. Но отправка логов плохо масштабируется.

Метрики – это сразу общая картина о состоянии приложения. Мы собираем не все детали, а только готовую выжимку: например, количество запросов к сервису. Картину мы получим, но в детали провалиться нельзя.

Push vs Pull

Push – принцип такой же, как с логами или базой данных: произошло событие – пишем данные в хранилище. События можно отправлять пачками. Способ прост в реализации с точки зрения клиента.

Pull – наоборот: приложения хранят в памяти компактные данные вроде обработано запросов: 25. Кто-то периодически приходит и собирает их, например по HTTP. С точки зрения клиента pull сложнее реализовать, но проще отлаживать

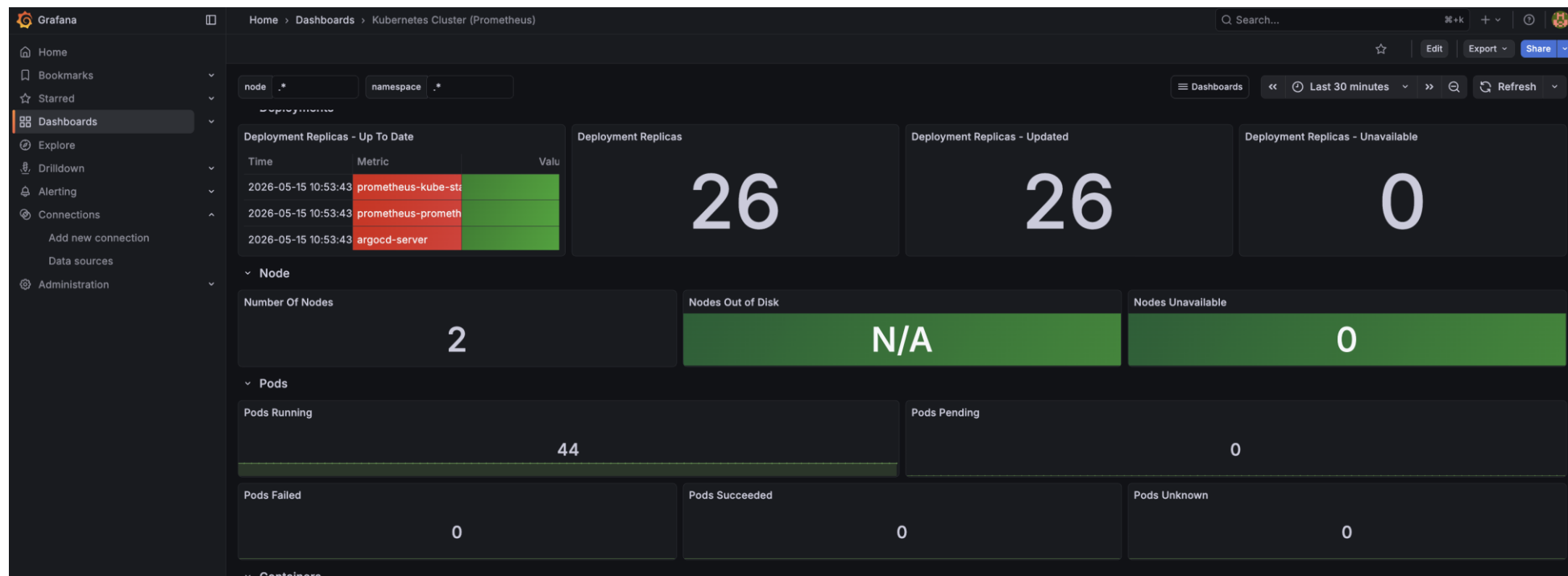
TSDB

Метрики нужно куда-то складывать и потом делать выборки. Эту задачу решают специализированные БД: Time Series Database.

TSDB нужны, чтобы сохранять **время и одно число**, привязанное к этому времени

Ежедневная температура: [(day1, t1), (day2, t2), ...]

Визуализация



Алерты

Самое полезное, что можно выжать из метрик и графиков — это алерты: нам не нужен живой человек, который постоянно следит за свободным местом на диске или за количеством ответов HTTP 500. Можно настроить автоматику, которая реагирует на выход графиков за допустимые границы и рассылает уведомления.

Scrape и подсчет метрик

- Задача приложения – выставить HTTP-страницу со своими метриками в определенном формате.
- Сервер периодически делает HTTP-запрос GET /metrics к нашему приложению.
- Ответ приложения сервер сохраняет в БД с текущим timestamp.
- Метрики приложения остаются в памяти приложения.

Безопасность

- Настроить аутентификацию в самом Prometheus
- Хостить endpoint /metrics на отдельном порту
- Сделать whitelist на уровне приложения

Prometheus – это

- Система мониторинга на основе метрик
- База данных, специализирующаяся под числовые значения
- По внутреннему таймеру периодически снимает показатели у экспортеров
- Подсистема обработки правил для алертов



Формат данных

```
1  # HELP http_requests_total Requests made to public API
2  # TYPE http_requests_total counter
3  http_requests_total{method="POST", url="/messages"} 1
4  http_requests_total{method="GET", url="/messages"} 3
5  http_requests_total{method="POST", url="/login"} 2
```

Counter

```
1  # Примеры метрик: количество обработанных запросов, ошибок, задач
2
3  http_requests_total{url="/login"} 10
4  http_requests_total{url="/"} 100
5
6  http_errors{status="500", url="/"} 3
7  http_errors{status="401", url="/"} 26
8  http_errors{status="400", url="/login"} 11
9  http_errors{status="404", url="/admin"} 298
10
11 jobs{type="cleanup", status="completed"} 42
12 jobs{type="cleanup", status="failed"} 38
```

Gauge

```
1  # Примеры метрик: количество обрабатываемых запросов прямо сейчас, занятая память, свободное место на диске
2
3  http_active_requests{app="web"} 5
4  http_active_requests{app="internal"} 1
5
6  memory_swap{host="test"} 0
7  memory_swap{host="prod"} 102400
8  memory_usage_bytes{host="test"} 1295007744
9  memory_usage_bytes{host="prod"} 5476434545
10
11 disk_free_bytes{path="/var/www/"} 29298077696
12 disk_free_bytes{path="/tmp/"} 37359484928
```

Histogram

```
1  # Пример метрики: распределение времени обработки HTTP-запросов по 4 бакетам
2
3  http_duration_bucket{url="/", le="0.1"} 100
4  http_duration_bucket{url="/", le="1"} 130
5  http_duration_bucket{url="/", le="5"} 140
6  http_duration_bucket{url="/", le="+Inf"} 141
7
8  http_duration_sum{url="/"} 152.7625769 # сумма всех значений, которые мы записали
9  http_duration_count{url="/"} 141 # количество значений
10
11
```

Простой запрос

`http_requests_total`

- один timestamp;
- лейблы какого-то ряда, который попал под запрос;
- значение из этого ряда в этот момент времени;
- лейблы второго ряда, который тоже попал под запрос;
- значение из второго ряда в этот же момент времени;
- ...и так далее для всех рядов.

`http_requests_total{job="prometheus",group="canary"}`

instant vector

Массив из key-value, где key – метрика с лейблами, а value — значение в запрошенный момент времени.

Один instant vector привязан к одному timestamp. Чтобы из этого получилось что-то полезное (для отображения на графике) — к запросам обычно добавляются диапазон времени и шаг, а в ответе возвращаются наборы instant vector-ов, попавшие под диапазон.

Метрика (временной ряд)	Время (unix timestamp)	Значение (double)
{__name__="http_requests_total",instance="app1"}	1615973700	0
{__name__="http_requests_total",instance="app2"}	1615973700	0
{__name__="http_requests_total",instance="app1"}	1615973730	0
{__name__="http_requests_total",instance="app2"}	1615973730	1
{__name__="http_requests_total",instance="app1"}	1615973760	1
{__name__="http_requests_total",instance="app2"}	1615973760	4
{__name__="http_requests_total",instance="app1"}	1615973790	3
{__name__="http_requests_total",instance="app2"}	1615973790	7
{__name__="http_requests_total",instance="app1"}	1615973820	4
{__name__="http_requests_total",instance="app2"}	1615973820	10

range vector

Массив из key-value, где key – метрика, а value – это значение в запрошенный момент времени + массив из [предыдущих значений за какой-то интервал]

```
1 http_requests_total{job="prometheus"}[5m]
2
```

http_requests_total{instance="app1"}[1m]

Время (unix timestamp)	Значение (double)	Предыдущие значения за 1 минуту
1615973700	0	[]
1615973730	0	[0]
1615973760	1	[0, 0]
1615973790	3	[0, 1]
1615973820	4	[1, 3]

http_requests_total{instance="app1"}[2m]

Время (unix timestamp)	Значение (double)	Предыдущие значения за 2 минуты
1615973700	0	[]
1615973730	0	[0]
1615973760	1	[0, 0]
1615973790	3	[0, 0, 1]
1615973820	4	[0, 0, 1, 3]

Offset

`http_requests_total offset 1d`

OR

`http_requests_total{app=~"apache|nginx|iis.*"}`

AND

`metric{tag=~"aaa.*", tag=~".*bbb"}`

Math

`metric{tag="value"} * 10 >= 50`

Cross

`metric1 and metric2{tag="something"}`

GROUP BY

sum(http_requests_total)

sum by (app, instance) (http_requests_total)

sum (http_requests_total) by (app, instance)

sum without (instance) (http_requests_total)

rate(range vector)

- считает скорость прироста в секунду (запросов/сек);
- подходит только для counter т.к. полагается на возрастание;
- учитывает сбросы метрики на 0, например при рестартах приложений;
- экстраполируется
- есть irate для резко прыгающих счетчиков.

```
1 rate(http_requests_total{app="nginx"}[5m])
```

delta(range vector)

- считает разницу за интервал (сколько запросов пришло);
- подходит только для gauge;
- экстраполируется

```
1 delta(ram_free{host="postgresql"}[5m])  
2
```

Персистентность

Ваши сервисы, которые выставляют метрики, не живут вечно — они рано или поздно обновятся или перезапустятся. Их метрики при этом сбросятся и начнут отсчитываться с нуля.

`rate()` учитывает такую ситуацию, но ожидает, что значение метрики всегда возрастает — без этого математика не сойдется. Поэтому совершенно нормально сделать счетчик «сколько приложение обработало запросов», и начинать его с нуля после рестарта.

Реплики приложения

```
1  # реплика 1:  
2  api_http_requests_total{method="POST", url="/login"} 2  
3  # реплика 2:  
4  api_http_requests_total{method="POST", url="/login"} 3  
5
```

Конфигурация

```
1  global:
2    scrape_interval: 15s
3    evaluation_interval: 15s
4
5  scrape_configs:
6    - job_name: prometheus
7      static_configs:
8        - targets: ['localhost:9090']
9
10   - job_name: 'prometheus_node_exporter'
11     scrape_interval: 5s
12     static_configs:
13       - targets: ['localhost:9100']
```

Домашнее задание

Спроектируйте систему мониторинга на базе Prometheus + Grafana для кластера Kubernetes, в котором работают долгоживущие микросервисы и batch-джобы Spark. Опишите не менее 5 ключевых метрик с лейблами, предложите не менее 3 дашбордов с примерами PromQL-запросов (например, для оценки здоровья системы, производительности Spark и бизнес-показателей), составьте не менее 4 правил алертинга с указанием условий, уровней серьезности и потенциально ложных срабатываний. Также решите проблему сбора метрик с короткоживущих Spark-джоб. Результат оформите в виде архитектурной документации